

Formal Verification Project Report

AHB-Lite SRAM Slave (Role B)

Engineer: Abid Hussain

Role: Role B (Formal Assertions on Slave DUT) **Target**

Protocol: AMBA 3 AHB-Lite Specification

Verification Engine: Cadence JasperGold FPV **Date:**

May 4, 2026

1. Executive Summary

This document presents the comprehensive formal verification results for the AHB-Lite SRAM Slave interface. The primary objective of this verification phase (Role B) was to mathematically prove that the Slave design strictly adheres to the ARM AMBA 3 AHB-Lite protocol specification, while simultaneously verifying the functional integrity of the internal SRAM memory array.

Using Cadence JasperGold, properties were compiled and analyzed during the final regression run. The verification strategy successfully exposed critical hardware bugs in the RTL design while proving the baseline structural protocol compliance.

2. Verification Methodology

2.1 Formal Property Verification (FPV)

Unlike dynamic simulation, which relies on generating pseudo-random stimulus across millions of clock cycles, Formal Property Verification (FPV) exhaustively proves whether a property can ever be violated under any legal input condition. By utilizing SystemVerilog Assertions (SVA), the protocol rules were codified into mathematical properties.

2.2 Symbolic Memory Tracking Architecture

Verifying memories formally typically results in state-space explosion because the tool attempts to track every bit of every address. To completely avoid this, a **Symbolic Memory Tracking** abstraction was utilized. The verification environment established a single, arbitrary symbolic address and tracked the two-phase AHB pipeline (Address Phase to Data Phase) exclusively for this address.

3. RTL Defect Tracking (Counterexamples)

The formal tool identified critical protocol violations within the DUT. Trace analysis revealed the following flaws directly mapping to our assertion failures:

BUG 1: Invalid Address Phase Capture (p_transfer_only_when_ready)

Observation: The slave is ignoring the incoming HREADY signal and starting the data phase early. The protocol dictates that a valid address phase is only captured when HREADY = 1. This early start creates a secondary failure regarding wait-state stability.

Actionable Fix: Gate the transfer capture condition in the RTL with the HREADY signal.

BUG 2: Wait State Protocol Violation (p_valid_wait_state_start)

Observation: The design asserts a wait state (pulling HREADYOUT low) without a preceding valid address phase. This is an illegal stalling of an idle bus, exacerbated by the failure to capture valid transfers properly.

Actionable Fix: Ensure HREADYOUT defaults to logic HIGH and is only pulled low when a valid NONSEQ or SEQ data phase is actively being processed.

4. Merged Formal Assertions Verification Results

The table below unifies the specified assertions, categorized by type, alongside root causes and actionable fixes derived from the verification trace analysis.

Assertion Name	Type	Spec Clause / Rule	Status	Root Cause / Observation	Actionable Fix / Notes
----------------	------	--------------------	--------	--------------------------	------------------------

p_reset	Protocol	Reset (Sec 7.1.2): Slave outputs must be OKAY	PASS	No issue. Reset behavior correct	None required
p_idle_busy_response	Protocol	Transfer Types (Sec 3.2): IDLE/ BUSY gets 0 wait-state	PASS	RAM behaves correctly	None required
p_no_x	Protocol	General: No unknown values on outputs	PASS	Outputs deterministic; No X propagation	None required
p_valid_wait_state_start	Protocol	Timing: Wait state must follow valid address phase	FAIL	Secondary failure due to early start / BUG FOUND	Fix p_transfer_only_when_ready first
p_two_cycle_error	Protocol	Error Response: ERROR must last exactly 2 cycles	PASS	No violation	None required
p_reset_recovery	Functional	Reset: System recovers after reset	PASS	System recovers; Stable behavior	None required
p_transfer_only_when_ready	Protocol	Basic Transfer: Transfer only when HREADY=1	FAIL	DUT ignores HREADY and starts early	Gate with HREADY in transfer condition
p_read_stable_no_write	Functional	Memory: Memory stable without write	PASS	Correct RAM behavior	None required

5. Formal Assumptions (Master Constraints)

To accurately simulate an AMBA AHB-Lite Master and prevent the formal tool from injecting illegal inputs, strict environmental constraints were implemented. Below is the detailed tracking table

Assumption ID	Status	AHB-Lite Rule Checked	Reference Figure / Section	Actionable Fix (TB Update)
assume_reset_htrans	Unreachable	Asynchronous Reset: HTRANS initializes to IDLE	Section 7.1.2	None required (Expected Formal Behavior)
assume_valid_htrans	Pass	Transfer Validity: HTRANS restricted to legal values	Section 3.2	None required
assume_valid_hburst	Pass	Burst Validity: HBURST restricted to 0-7	Section 3.5	None required
assume_valid_hsize	Pass	Size Validity: HSIZE restricted to 32-bit bounds	Section 3.4	None required
assume_valid_hwrite	Pass	Signal Integrity: HWRITE has no X/Z states	Section 3.1	None required
assume_valid_hsel	Pass	Signal Integrity: HSEL has no X/Z states	Section 4.1	None required
assume_valid_hready	Pass	Signal Integrity: HREADY has no X/Z states	Section 3.1	None required
assume_valid_haddr	Pass	Signal Integrity: HADDR valid during active transfer	Section 3.1	None required
assume_align_halfword	Pass	Address Alignment: Halfword aligned to 2-byte	Section 3.4	None required
assume_align_word	Pass	Address Alignment: Word aligned to 4-byte	Section 3.4	None required
assume_1kb_boundary	Pass	Boundary Limit: Bursts must not cross 1KB boundary	Section 4.1	None required
assume_stable_haddr_wait	Pass	Wait-State Stability: HADDR held during HREADY=0	Section 3.1	None required
assume_stable_htrans_wait	Pass	Wait-State Stability: HTRANS held during HREADY=0	Section 3.1	None required
assume_stable_hsize_wait	Pass	Wait-State Stability: HSIZE held during HREADY=0	Section 3.1	None required
assume_stable_hburst_wait	Pass	Wait-State Stability: HBURST held during HREADY=0	Section 3.1	None required
assume_stable_hwrite_wait	Pass	Wait-State Stability: HWRITE held during HREADY=0	Section 3.1	None required
assume_stable_hsel_wait	Pass	Wait-State Stability: HSEL held during HREADY=0	Section 4.1	None required
assume_no_seq_after_idle	Pass	Protocol Flow: SEQ cannot immediately follow IDLE	Section 3.2	None required
assume_valid_seq_pred	Pass	Protocol Flow: Valid predecessors for SEQ phase	Section 3.2	None required
assume_valid_busy_pred	Pass	Protocol Flow: Valid predecessors for BUSY phase	Section 3.2	None required
assume_single_no_seq	Pass	Protocol Flow: SINGLE burst forbids SEQ transfers	Section 3.5	None required
assume_single_no_busy	Pass	Protocol Flow: SINGLE burst forbids BUSY transfers	Section 3.5	None required
assume_const_hsize_burst	Pass	Burst Integrity: Constant HSIZE during burst	Section 3.5	None required
assume_const_hwrite_burst	Pass	Burst Integrity: Constant HWRITE during burst	Section 3.5	None required
assume_const_hburst_burst	Pass	Burst Integrity: Constant HBURST during burst	Section 3.5	None required
assume_burst_len_incr4	Pass	Burst Length: INCR4 executes exactly 4 beats	Section 3.5	None required
assume_burst_len_wrap4	Pass	Burst Length: WRAP4 executes exactly 4 beats	Section 3.5	None required
assume_burst_len_incr8	Pass	Burst Length: INCR8 executes exactly 8 beats	Section 3.5	None required
assume_burst_len_wrap8	Pass	Burst Length: WRAP8 executes exactly 8 beats	Section 3.5	None required
assume_burst_len_incr16	Pass	Burst Length: INCR16 executes exactly 16 beats	Section 3.5	None required
assume_burst_len_wrap16	Pass	Burst Length: WRAP16 executes exactly 16 beats	Section 3.5	None required
assume_incr_addr_math	Pass	Burst Address: INCR strictly increments by HSIZE	Section 3.5	None required
assume_busy_addr_stable	Pass	Protocol Flow: BUSY phase maintains stable address	Section 3.2	None required
assume_wrap4_word_bound	Pass	Burst Boundary: WRAP4 respects 16-byte wrap limit	Section 3.5	None required
assume_wrap8_word_bound	Pass	Burst Boundary: WRAP8 respects 32-byte wrap limit	Section 3.5	None required
assume_wrap16_word_bound	Pass	Burst Boundary: WRAP16 respects 64-byte wrap limit	Section 3.5	None required
assume_wrap4_hw_bound	Pass	Burst Boundary: WRAP4 Halfword respects 8-byte limit	Section 3.5	None required

6. Coverage and Reachability Analysis

To guarantee that the formal environment actively exercised complex DUT states (preventing vacuous assertion passes), 90 Cover properties were instrumented. The suite achieved an excellent **92.2% reachability hit rate**, proving that the tool deeply explored sequences like back- to-back wait states, wrapping bursts, and specific byte-lane interactions.

7. Conclusion and Next Steps

The verification suite successfully verified standard AMBA 3 protocol compliance for reset, X-propagation, and basic memory responses. However, the DUT is currently failing fundamental operational timing requirements.

Action Items for RTL Design (Role A):

- Update the transition logic from the Address Phase to the Data Phase to explicitly check that `HREADY == 1`.
- Ensure `HREADYOUT` defaults to logic HIGH and only stalls the bus legitimately.